⌂ Volver al inicio

Índice

1. Introducción y objetivos
2. Entornos integrados de desarrollo
    - Actividades 2
3. Identificación de los elementos de un programa
    - Actividades 3
4. Estructura y bloques fundamentales
    - Actividades 4
5. Variables
    - Actividades 5
6. Tipos de datos
7. Literales
8. Constantes
    - Actividades 8
9. Operadores y expresiones
    - Actividades 9
10. Conversiones de tipo
11. Comentarios
12. Videotutoriales y recursos

---

1. Introducción y objetivos

↑ Volver al índice



Python es un lenguaje de programación de propósito general, interpretado, de alto nivel y con una sintaxis diseñada para favorecer la legibilidad. Se utiliza en automatización, desarrollo web, análisis de datos, inteligencia artificial, ciencia, educación, administración de sistemas y creación de herramientas de escritorio.

Este documento presenta los fundamentos necesarios para leer, escribir, ejecutar, probar, depurar y documentar programas en Python 3. Los ejemplos se han pensado para poder copiarse en un archivo `.py` y ejecutarse con una instalación estándar de Python, sin depender de librerías externas.

Contenidos clave:

- Entornos integrados de desarrollo
- Variables
- Tipos de datos
- Literales
- Constantes
- Operadores y expresiones
- Conversiones de tipo
- Comentarios
- Estructuras de control
- Selección
- Repetición
- Salto
- Control de excepciones
- Depuración
- Depurador
- Documentación

Objetivos de aprendizaje

Al terminar el tema, se debería poder:

- Reconocer las partes de un programa y explicar la función de cada una.
- Crear variables y seleccionar tipos de datos adecuados.
- Distinguir entre literales, variables y constantes convencionales.
- Construir expresiones usando operadores aritméticos, relacionales y lógicos.
- Convertir datos de un tipo a otro y validar entradas.
- Escribir comentarios y documentación útil.
- Utilizar un IDE para editar, ejecutar y depurar programas.
- Tomar decisiones con `if`, `elif` y `else`.
- Repetir acciones con `for` y `while`.
- Alterar el flujo con `break`, `continue`, `pass` y `return`.
- Controlar errores mediante excepciones.
- Investigar errores con mensajes, pruebas y puntos de interrupción.
- Organizar un pequeño proyecto mantenible.

Requisito: Python 3

Los ejemplos están escritos para Python 3. En una terminal se puede comprobar la versión con:

```
python --version
```

En algunos sistemas el comando es:

```
python3 --version
```

El primer programa habitual es:

```
print("Hola, Python")
```

`print()` es una función que escribe información en la salida estándar. La cadena está delimitada por comillas y la llamada se ejecuta al escribir los paréntesis.

---

2. Entornos integrados de desarrollo

⬆ Volver al índice

Un IDE (Integrated Development Environment, entorno integrado de desarrollo) reúne herramientas para escribir, ejecutar y mantener programas. Para Python, opciones habituales son Visual Studio Code, PyCharm, IDLE y JupyterLab.

2.1 Componentes de un IDE

Un IDE suele incluir:

- Editor con resaltado de sintaxis.
- Autocompletado y navegación de símbolos.
- Terminal integrada.
- Ejecución y configuración de programas.
- Depurador con puntos de interrupción.
- Integración con Git.
- Diagnósticos de sintaxis y tipos.
- Herramientas de formato y análisis estático.
- Explorador de archivos y gestión de entornos.

2.2 Flujo de trabajo en Visual Studio Code

Un flujo básico es:

1. Abrir la carpeta del proyecto.
2. Instalar la extensión de Python de Microsoft.
3. Seleccionar un intérprete con `Python: Select Interpreter`.
4. Crear un archivo como `main.py`.
5. Escribir el programa y guardarlo.
6. Ejecutarlo desde el botón de ejecución o la terminal.
7. Revisar diagnósticos y corregirlos.
8. Ejecutar pruebas y depurar con puntos de interrupción.

2.3 Entornos virtuales

Un entorno virtual aísla dependencias de un proyecto:

```
python -m venv .venv
```

Activación en Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Activación en macOS o Linux:

```
source .venv/bin/activate
```

Después se pueden instalar paquetes sin afectar a otras aplicaciones:

```
python -m pip install nombre-paquete
```

Es buena práctica guardar dependencias:

```
python -m pip freeze > requirements.txt
```

2.4 Formateo y análisis

Herramientas populares son `black` para formato y `ruff` para análisis y estilo. Si se utilizan en un proyecto, deben configurarse y ejecutarse de forma consistente. El objetivo no es obedecer mecánicamente a una herramienta, sino reducir errores y discusiones de estilo.

2.5 Ejecutar desde la terminal

```
python main.py
```

Ejecutar mediante `python -m` es útil para módulos y paquetes:

```
python -m mi_paquete
```

Actividades 2

Actividad 1: preparar un entorno

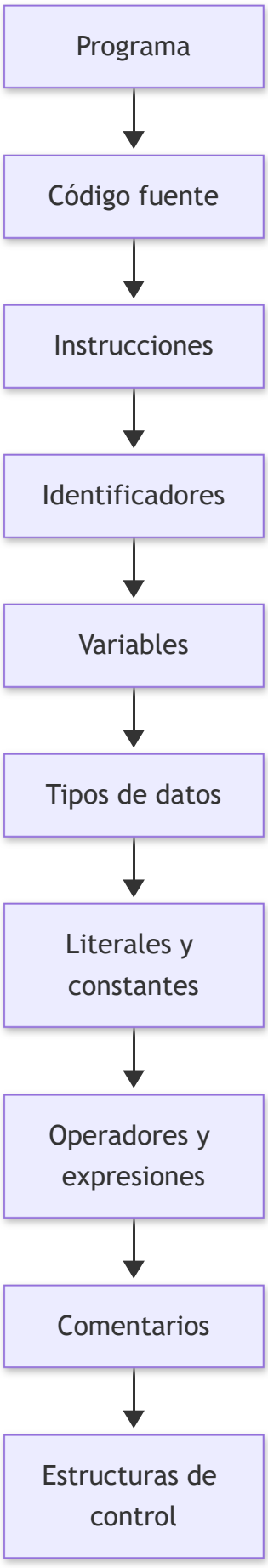
Crea una carpeta de proyecto, genera `.venv`, selecciona el intérprete en el IDE, crea `main.py`, ejecuta un saludo y documenta en un `README.md` los pasos de instalación y ejecución.

---

3. Identificación de los elementos de un programa informático

⬆ Volver al índice

Un programa es un conjunto ordenado de instrucciones que procesa datos para producir resultados. Aunque Python permite escribir programas muy pequeños, incluso un ejemplo de pocas líneas contiene varios elementos conceptuales.



3.1 Código fuente

El código fuente es el texto que escribe la persona programadora. En Python suele guardarse en archivos con extensión `.py`:

```
nombre = input("¿Cómo te llamas? ")
mensaje = f"Hola, {nombre}"
print(mensaje)
```

El intérprete de Python lee ese código, comprueba su sintaxis y ejecuta las instrucciones. El código fuente debe ser comprensible para las personas y válido para el intérprete.

3.2 Instrucciones

Una instrucción es una orden que puede producir un efecto. Algunos ejemplos son una asignación, una llamada a función o una sentencia condicional:

```
total = 25 + 10       # asignación
print(total)          # llamada a función

if total > 30:        # selección
    print("Supera 30")
```

Python utiliza el salto de línea para separar normalmente las instrucciones. También se puede usar un punto y coma, pero no es recomendable porque reduce la legibilidad.

Las tabulaciones (o su equivalente en espacios) no separan instrucciones, sino que definen la indentación: el nivel de sangrado que delimita los bloques de código (por ejemplo, el cuerpo de un `if`, un `for` o una función). A diferencia de otros lenguajes que usan llaves `{}`, en Python la indentación es obligatoria y forma parte de la sintaxis. Se recomienda usar siempre 4 espacios y no mezclar tabulaciones con espacios, ya que Python puede rechazar el código con un error de indentación (`IndentationError` o `TabError`).

3.3 Identificadores

Los identificadores son nombres que designan variables, funciones, clases, módulos u otros elementos. Sus reglas principales son:

- Pueden contener letras, dígitos y guiones bajos.
- No pueden comenzar por un dígito.
- Distinguen mayúsculas de minúsculas: `total`, `Total` y `TOTAL` son nombres distintos.
- No deben coincidir con palabras reservadas como `if`, `for`, `class` o `return`.
- Se recomienda `snake_case` para variables y funciones.

```
precio_unitario = 12.50
cantidad = 4
importe = precio_unitario * cantidad
```

Conviene elegir nombres que expresen el significado: `edad_usuario` es preferible a `x` cuando la variable representa una edad.

3.4 Palabras reservadas

Python reserva determinadas palabras para su propia sintaxis. Se pueden consultar con:

```
import keyword

print(keyword.kwlist)
```

Entre ellas están `and`, `as`, `assert`, `break`, `class`, `continue`, `def`, `elif`, `else`, `except`, `False`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `None`, `nonlocal`, `not`, `or`, `pass`, `raise`, `return`, `True`, `try`, `while`, `with` y `yield`.

El listado completo y actualizado de palabras reservadas de Python 3 puede consultarse en la documentación oficial de Python.

3.5 Entrada, procesamiento y salida

Un esquema muy común es **entrada -> procesamiento -> salida**:

```
# Entrada
base = float(input("Base del rectángulo: "))
altura = float(input("Altura del rectángulo: "))

# Procesamiento
area = base * altura

# Salida
print(f"El área es {area:.2f}")
```

- **Entrada:** datos recibidos desde teclado, archivos, red, sensores o parámetros.
- **Procesamiento:** cálculos, comparaciones, transformaciones y decisiones.
- **Salida:** información mostrada, guardada o enviada a otro sistema.

3.6 Sintaxis y semántica

La **sintaxis** son las reglas que definen si el texto de un programa está bien formado, es decir, si el intérprete es capaz de leerlo. Determina cosas como el orden de las palabras clave, el uso de paréntesis, comillas o dos puntos, y la indentación. Un fallo de sintaxis impide que el programa llegue a ejecutarse: el intérprete se detiene y muestra un `SyntaxError` antes de procesar ninguna instrucción.

La **semántica** es el significado de lo que está escrito, es decir, qué hace realmente cada instrucción cuando se ejecuta. Un programa puede tener una sintaxis perfectamente correcta y aun así hacer algo distinto de lo que se pretendía, porque el significado de las instrucciones no coincide con la intención de quien programa.

La diferencia se puede resumir así:

- La sintaxis responde a la pregunta "¿está bien escrito?".
- La semántica responde a la pregunta "¿hace lo que se quiere?".
- Un error de sintaxis se detecta antes de ejecutar el programa.
- Un error de semántica (o error lógico) solo se detecta observando el comportamiento o el resultado durante o después de la ejecución.

```
print("Hola")       # sintaxis y significado correctos
```

El siguiente ejemplo tiene un error sintáctico porque falta cerrar el paréntesis; el intérprete no consigue leer la instrucción:

```
# print("Hola"
```

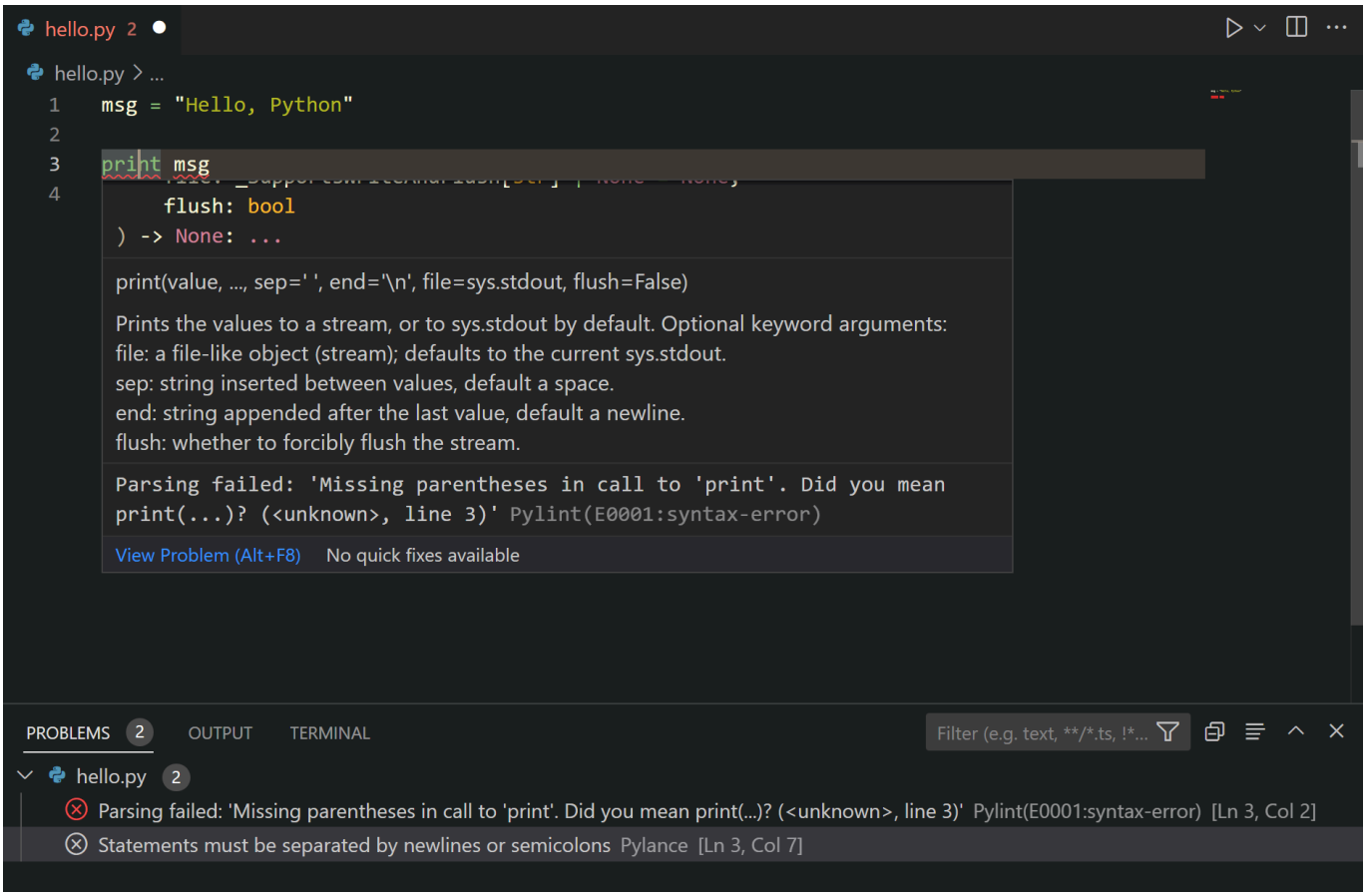
Este ejemplo es sintácticamente correcto (el intérprete lo ejecuta sin protestar), pero tiene un error semántico si la intención era sumar dos cantidades y multiplicar el número 10 tres veces produce un resultado distinto del esperado:

```
resultado = "10" * 3
print(resultado)    # produce "101010", no 30
```

3.7 Errores de un programa

Los errores más habituales son:

1. **Errores de sintaxis:** el código no respeta las reglas del lenguaje.
2. **Errores de ejecución:** el programa empieza, pero falla durante la ejecución, por ejemplo al dividir entre cero.
3. **Errores lógicos:** el programa se ejecuta, pero produce un resultado incorrecto.



En Visual Studio Code, los errores y avisos se subrayan directamente en el código (normalmente en rojo los errores y en amarillo los avisos) y también aparecen listados en el panel **Problems**, donde se puede ver el mensaje completo, el archivo y la línea exacta en la que se producen.

```
# Sintaxis: falta el signo de cierre.
# print("Hola"

# Ejecución: ZeroDivisionError.
# resultado = 10 / 0

# Lógica: la fórmula no corresponde al perímetro de un cuadrado.
lado = 5
perimetro_incorrecto = lado * lado
```

Actividades 3

Actividad 1: reconocer elementos

Analiza el siguiente código e identifica variables, literales, operadores, funciones, entrada, procesamiento y salida:

```
precio = float(input("Precio: "))
unidades = int(input("Unidades: "))
descuento = 0.10
```

```
total = precio * unidades * (1 - descuento)
print(f"Total: {total:.2f}")
```

Actividad 2: identificadores válidos

Indica cuáles de los siguientes nombres son identificadores válidos en Python y, para los que no lo sean, explica por qué: `total_1`, `1total`, `Total`, `for`, `_precio`, `precio-unitario`, `precio_unitario_€`, `class`, `Class`, `__init__`, `2do_intento`, `nombre completo`, `número_de_cuenta`, `total__final`, `def_valores`, `True`, `variable-2`, `_`, `mi.variable`, `while1`.

Actividad 3: entrada, procesamiento y salida

Observa el siguiente código y clasifica cada línea como entrada, procesamiento o salida escribiéndolo en un comentario al lado. Algunas líneas combinan más de una categoría y una de ellas no encaja en ninguna de las tres:

```
import math

nombre = input("¿Cómo te llamas? ")
edad = int(input("¿Qué edad tienes? "))
radio = float(input("Radio del círculo: "))

mayor_edad = edad >= 18
recargo = 0.05 if not mayor_edad else 0.0
superficie = math.pi * radio ** 2
superficie_con_recargo = superficie * (1 + recargo)

mensaje = f"{nombre}, ¿eres mayor de edad? {mayor_edad}"

print(mensaje)
print(f"Superficie: {superficie:.2f}")
print(f"Superficie con recargo: {superficie_con_recargo:.2f}")
```

Actividad 4: depurar tipos de error

El siguiente código mezcla varios problemas: errores de sintaxis, de indentación/tabulación, de ejecución y errores lógicos. Localízalos todos, indica de qué tipo es cada uno y corrígelos:

```
def calcular_media(numeros):
    suma = 0
  for numero in numeros:
        suma = suma + numero
    media = suma / len(numeros)
    return media


def calcular_area_triangulo(base, altura)
```

```
    area = base * altura / 2
    return area


notas = [4, 6, 8, 10]
print("Media: " media)
print("Área:", calcular_area_triangulo(4, 3))

lista_vacia = []
print("Media lista vacía:", calcular_media(lista_vacia))
```

Pistas:

- Hay una línea que mezcla espacios y tabulaciones en el mismo bloque.
- Hay un bloque indentado con menos sangrado del que le corresponde.
- Falta un símbolo indispensable en la definición de una función.
- Una llamada a `print` referencia una variable que no existe con ese nombre.
- Una de las funciones falla con una entrada concreta aunque el código sea sintácticamente correcto.

Actividad 5: esquema propio

A partir del esquema de la sección 3, elabora tu propio diagrama (en papel o en Mermaid) que represente los elementos de un programa distinto, por ejemplo uno que calcule el precio final de una compra con IVA. Incluye al menos: identificadores, tipos de datos, literales, operadores y estructuras de control.

---

4. Estructura y bloques fundamentales

⬆ Volver al índice

Python utiliza la indentación para delimitar bloques de código. Esto diferencia a Python de lenguajes que emplean llaves, y hace que la presentación visual del programa forme parte de su sintaxis.

4.1 Indentación

Después de una línea que termina en dos puntos, las instrucciones del bloque se escriben indentadas:

```
edad = 20

if edad >= 18:
    print("Es mayor de edad")
    print("Puede continuar")
```

La recomendación habitual es usar cuatro espacios por nivel. No se deben mezclar tabuladores y espacios en el mismo bloque.

```
if True:
    mensaje = "Este bloque tiene cuatro espacios"
```

```
    print(mensaje)
```

4.2 Bloques anidados

Un bloque puede contener otro bloque. Cada nivel añade una indentación:

```
temperatura = 26
llueve = False

if temperatura > 20:
    if llueve:
        print("Temperatura agradable y lluvia")
    else:
        print("Temperatura agradable y tiempo seco")
```

Cuando hay muchos niveles anidados, suele ser mejor reorganizar la lógica con condiciones compuestas o funciones.

4.3 Módulos y función `main`

> **Idea clave:** un archivo Python puede ser un programa que se ejecuta o un módulo que otro archivo importa. `main()` organiza la ejecución y evita que la interacción se dispare accidentalmente al importar.

Programa frente a módulo

Uso	Qué ocurre	Ejemplo
Programa	Se ejecuta directamente y realiza una tarea completa.	`python saludos.py`
Módulo	Otro archivo lo importa para reutilizar sus funciones.	`from saludos import saludar`

La ventaja es escribir la lógica una sola vez y reutilizarla desde una aplicación, una prueba automática o el intérprete interactivo.

El problema de ejecutar código al importar

Este archivo `saludos.py` parece correcto si se ejecuta directamente:

```
def saludar(nombre):
    return f"Hola, {nombre}"


nombre = input("Nombre: ")
print(saludar(nombre))
```

Pero si `programa.py` solo quiere reutilizar la función:

```
from saludos import saludar

print(saludar("Ana"))
```

Python también ejecutará el `input()` de `saludos.py` durante la importación. Aparecerá una pregunta por teclado antes de `Hola, Ana`, aunque no la hemos solicitado.

La solución: separar definición y ejecución

Colocamos la interacción dentro de `main()` y protegemos su llamada:

```
def saludar(nombre):
    return f"Hola, {nombre}"


def main():
    nombre = input("Nombre: ")
    print(saludar(nombre))


if __name__ == "__main__":
    main()
```

> **Lectura de la condición:** ejecuta `main()` únicamente si este archivo es el programa principal que se ha ejecutado directamente.

Situación	Valor de `__name__`	¿Se llama a `main()`?
`python saludos.py`	`"__main__"`	Sí
`programa.py` importa `saludos`	`"saludos"`	No

Flujo completo entre dos archivos

`saludos.py` contiene las funciones y su punto de entrada:

```
def saludar(nombre, tratamiento="Hola"):
    return f"{tratamiento}, {nombre}"


def main():
    nombre = input("Nombre: ")
    print(saludar(nombre))


if __name__ == "__main__":
    main()
```

`programa.py` reutiliza la función:

```
from saludos import saludar


def main():
    print(saludar("Ana"))
    print(saludar("Luis", tratamiento="Buenos días"))


if __name__ == "__main__":
    main()
```

Al ejecutar `python programa.py`:

1. Se cargan las funciones de `saludos.py`.
2. No se ejecuta su `main()`, porque `__name__` vale `"saludos"`.
3. Se ejecuta el `main()` de `programa.py`, que sí es el archivo principal.

Reparto de responsabilidades

- **Funciones:** tareas concretas y reutilizables.
- **`main()`:** coordinación de entrada, llamadas y salida.
- **Condición `if`:** único punto que inicia automáticamente el programa.
- **Nivel superior del módulo:** definiciones y constantes, no acciones inesperadas.

> **Error frecuente:** escribir `main()` directamente al final, sin la condición. Así la interacción se inicia cada vez que otro archivo importa el módulo.

4.4 Funciones como bloques reutilizables

Una función agrupa instrucciones con un nombre, recibe parámetros y puede devolver un resultado. Es una forma de convertir una tarea compleja o repetida en un bloque que podemos llamar desde varios lugares.

```
def calcular_iva(precio, porcentaje=21):
    """Devuelve el importe del IVA para un precio dado."""
    return precio * porcentaje / 100


iva = calcular_iva(100)
print(iva)
```

Las funciones reducen la duplicación, facilitan las pruebas y permiten dividir un problema grande en tareas pequeñas.

4.5 Alcance de los nombres

Una variable creada dentro de una función suele ser local a esa función:

```
def crear_mensaje():
    mensaje = "Solo existe dentro de la función"
    return mensaje


resultado = crear_mensaje()
print(resultado)
```

Es preferible pasar datos como argumentos y devolver resultados, en lugar de depender de variables globales.

4.6 Nombres de archivos de programas

Los archivos de Python deben guardarse con la extensión `.py`. Para que sean fáciles de localizar y entender, se recomienda:

- escribir los nombres en minúsculas;
- separar las palabras con guiones bajos (`snake_case`);
- incluir, cuando sea útil, una referencia al ejercicio o a la tarea que resuelve el programa.

Por ejemplo:

```
operaciones_1.py
operaciones_media_numeros.py
gestor_tareas.py
```

Es preferible evitar espacios, acentos y caracteres especiales en los nombres de archivo, ya que pueden causar dificultades al ejecutar programas desde la terminal o al compartirlos entre sistemas.

La plantilla con `main()` mostrada en la sección anterior y esta convención de nombres siguen las recomendaciones de Programa básico de Python de mclibre.org.

4.7 Cómo se ejecuta un programa Python

Un programa Python se procesa de arriba abajo. Esta idea explica muchos errores de principiante: una función debe estar definida antes de llamarse, una variable debe recibir un valor antes de utilizarse y una condición se comprueba en el punto exacto en el que aparece. El intérprete no reordena las instrucciones.

En un programa pequeño podemos imaginar el flujo así:

1. Python lee una instrucción.
2. Evalúa las expresiones que necesita.
3. Ejecuta la acción resultante.
4. Continúa con la siguiente instrucción, salvo que encuentre una decisión, un bucle, una llamada a función o una excepción.

```
print("1. Inicio")

def duplicar(numero):
    return numero * 2


valor = duplicar(4)
print("2. Resultado:", valor)
print("3. Fin")
```

La definición de `duplicar` no ejecuta todavía su cuerpo: crea la función. El cuerpo se ejecuta cuando aparece `duplicar(4)`. Distinguir entre definir y llamar es fundamental para organizar programas en funciones.

4.8 La indentación como parte de la sintaxis

En muchos lenguajes las llaves indican dónde empieza y termina un bloque. En Python esa información la aporta la indentación. No es solo formato visual: cambiar el sangrado puede cambiar el significado del programa. Todas las instrucciones del mismo bloque deben comenzar en la misma columna.

```
numero = 7

if numero > 0:
    print("El número es positivo")
    if numero % 2 == 0:
        print("Además es par")
    else:
        print("Además es impar")
```

En el ejemplo hay dos niveles: el primer `if` contiene al segundo, y este último contiene dos posibles bloques. Un `else` se relaciona con el `if` que está al mismo nivel de indentación. Para evitar errores:

- configura el editor para insertar cuatro espacios al pulsar Tab;
- no mezcles tabuladores y espacios;
- mantén el mismo nivel para instrucciones hermanas;

- reduce la profundidad extrayendo una función cuando un bloque sea difícil de leer.

4.9 Diseñar un bloque antes de escribirlo

Antes de codificar conviene expresar cada bloque como una responsabilidad. Un programa que calcula una compra puede dividirse en `leer_precio()`, `calcular_total()`, `mostrar_resumen()` y `main()`. Así no se mezclan entrada, cálculos y salida, y se puede probar el cálculo sin responder preguntas por teclado. La regla práctica es que una función debería poder describirse con un verbo: calcular, validar, convertir, buscar o mostrar.

```
def calcular_total(precio, unidades, iva=21):
    subtotal = precio * unidades
    importe_iva = subtotal * iva / 100
    return subtotal + importe_iva


def main():
    precio = 12.5
    unidades = 3
    print(f"Total: {calcular_total(precio, unidades):.2f} euros")


if __name__ == "__main__":
    main()
```

`main()` coordina el programa, pero no debe convertirse en un lugar donde se acumule toda la lógica. Si aparece una nueva tarea, normalmente puede extraerse como otra función pequeña y comprobable.

4.10 Parámetros, argumentos y valores devueltos

El parámetro es el nombre de la definición; el argumento es el valor concreto de la llamada. En `calcular_total(precio, unidades)`, los parámetros son `precio` y `unidades`; en `calcular_total(12.5, 3)`, los argumentos son `12.5` y `3`.

`return` termina la función y entrega un valor. `print()` solo muestra algo en pantalla y no sustituye a `return`:

```
def sumar_y_devolver(a, b):
    return a + b


def sumar_y_mostrar(a, b):
    print(a + b)


resultado = sumar_y_devolver(2, 3)  # vale 5
otro_resultado = sumar_y_mostrar(2, 3)  # muestra 5, pero vale None
```

Una función que calcula debería devolver el resultado; quien la llama decidirá si lo muestra, lo guarda o lo utiliza en otra operación. Esta separación permite reutilizar el código en una consola, una prueba automática o una interfaz.

4.11 Alcance, flujo y errores habituales

El alcance indica dónde se puede utilizar un nombre. Los nombres locales nacen dentro de una función y dejan de estar disponibles al terminar su ejecución. Una variable global puede ser visible en más lugares, pero depender demasiado de ella hace que las funciones sean difíciles de entender y probar.

```
def crear_usuario():
    nombre = "Lucía"
    return nombre


usuario = crear_usuario()
# print(nombre)  # NameError: solo existe dentro de crear_usuario
```

Los errores más frecuentes al construir bloques son olvidar los dos puntos, indentar una línea en el nivel incorrecto, llamar a una función con argumentos incorrectos, confundir `print()` con `return` y ejecutar código de prueba al importar un módulo por no usar `main()`. Para localizar el problema, prueba primero cada función con datos sencillos, comprueba qué recibe y qué devuelve, y añade complejidad solo después de validar el bloque pequeño.

4.12 Videotutoriales recomendados

Aquí algunos videotutoriales en español:

- Indentación y bloques en Python.
- Funciones, parámetros y `return` en Python.
- Módulos y `if __name__ == "__main__"`.
- Alcance de variables y funciones en Python.

Actividades 4

> **Nivel inicial:** estas actividades no piden crear un programa desde cero. Primero se observa, se ordena, se completa y se explica.

Actividad 1: reconocer los bloques

Observa este ejemplo y colorea o marca cada parte con una letra:

```
def mostrar_mensaje():       # A
    mensaje = "Bienvenido"    # B
    print(mensaje)             # C
```

```
mostrar_mensaje()           # D
```

Relaciona cada letra con una descripción:

- definición de una función;
- instrucción que guarda un texto;
- instrucción que muestra un resultado;
- llamada que hace que la función se ejecute.

Después responde: ¿qué líneas se agrupan por la indentación? ¿Qué ocurriría si la línea `print(mensaje)` se escribiera sin sangrado?

Actividad 2: ordenar un programa

Las siguientes tarjetas forman un programa, pero están desordenadas. Numéralas del 1 al 6 para indicar el orden correcto. No es necesario escribir código.

```
[ ] Mostrar el saludo en pantalla.
[ ] Definir una función llamada saludar.
[ ] Pedir el nombre de la persona.
[ ] Devolver el texto "Hola, " seguido del nombre.
[ ] Llamar a la función saludar.
[ ] Terminar el programa.
```

Explica por qué no tendría sentido llamar a `saludar` antes de definirla. Como ampliación, dibuja flechas entre las tarjetas que dependen unas de otras.

Actividad 3: leer la indentación

En ambos fragmentos, `hay_entradas` vale `True`. Los dos fragmentos son válidos, pero la última instrucción no pertenece al mismo bloque en ambos casos. Para cada fragmento, indica qué mensajes se muestran y explica por qué usando las palabras **bloque**, **nivel** e **indentación**.

```
# Fragmento A
if hay_entradas:
    print("Comenzamos")
    print("Hay trabajo")
```

```
# Fragmento B
if hay_entradas:
    print("Comenzamos")
print("Hay trabajo")
```

Después marca las instrucciones que pertenecen al `if` en cada fragmento. Recuerda: una instrucción indentada bajo el `if` pertenece a su bloque; una instrucción que vuelve al margen izquierdo está fuera de él.

Actividad 4: programa o módulo

Una pequeña empresa de asistencia informática quiere un programa de consola para preparar el presupuesto de una reparación. El programa debe pedir el nombre del cliente, las horas de trabajo, el precio por hora, el coste de las piezas y el porcentaje de IVA. Después debe calcular el coste de mano de obra, el subtotal, el IVA y el total, y mostrar un presupuesto desglosado.

Divide el problema en módulos `.py`. Propón al menos tres módulos e indica la responsabilidad de cada uno. Puedes considerar, por ejemplo, un módulo para coordinar el programa, otro para los cálculos y otro para solicitar o mostrar información. No existe una única división correcta: lo importante es que cada módulo tenga una responsabilidad clara y que el programa principal coordine el trabajo.

Completa una tabla como esta:

Módulo propuesto	Responsabilidad	Funciones que podría contener
`main.py`	_____	_____
_____	_____	_____
_____	_____	_____

Al terminar, explica por qué no conviene escribir todo el programa en un solo archivo. Los módulos de esta actividad son archivos Python que contienen código; no se pide guardar datos en ficheros ni usar bases de datos.

Actividad 5: extraer funciones de un enunciado

Un taller de asistencia informática quiere un programa que prepare el presupuesto de una reparación. El programa debe pedir el nombre del cliente, las horas de trabajo, el precio por hora, el coste de las piezas y el porcentaje de IVA. A continuación, debe calcular el coste de la mano de obra, el subtotal, el importe del IVA y el total. Por último, debe mostrar el nombre del cliente y el presupuesto desglosado.

Sin escribir todavía el código, analiza el enunciado y divídelo en funciones. Completa una fila por cada función que consideres necesaria:

Nombre propuesto para la función	Qué tarea realiza	Qué datos necesita	Qué resultado devuelve o muestra
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

Después, indica qué función coordinaría el proceso completo y escribe el orden en que llamaría a las demás. Puede haber distintas soluciones: cada función debe encargarse de una tarea concreta y el conjunto debe

resolver todo el enunciado.

Actividad 6: completar una plantilla con pistas

Completa los seis huecos, identificados con letras, usando estas palabras: `def`, `main`, `return`, `if` y `print`. Algunas palabras se necesitan más de una vez.

```
_____ saludar(nombre):  # A
    _____ f"Hola, {nombre}"  # B


_____ _____():  # C, D
    _____ (saludar("Ana"))  # E


_____ __name__ == "__main__":  # F
    main()
```

Después de completar la plantilla:

1. Señala la definición de cada función y el valor que devuelve `saludar()`.
2. Si ejecutas el archivo directamente, indica qué función se llama desde el bloque `if`.
3. Sigue la llamada a `saludar("Ana")` y escribe el texto que termina mostrando `print()`.

Actividad 7: diseñar las llamadas entre módulos

Usa la propuesta de módulos de la actividad 4 de «Actividades 3» para diseñar cómo colaboran. Escribe el nombre de una función que podría realizar cada tarea y qué información recibe o devuelve:

Tarea	Función propuesta	Información que recibe o devuelve
Pedir los datos del presupuesto	_____	_____
Calcular el coste de mano de obra	_____	_____
Calcular el IVA y el total	_____	_____
Mostrar el presupuesto	_____	_____

Después dibuja un esquema con flechas que muestre qué módulos llaman a las funciones de otros. El flujo debe comenzar en `main.py`; indica qué función coordina las llamadas y en qué orden se prepara y se muestra el presupuesto. No hace falta escribir el programa completo.

---

5. Variables

⬆ Volver al índice

Una variable es un nombre que permite guardar y utilizar un valor durante la ejecución de un programa. En Python, una variable se crea al asignarle un valor por primera vez.

Variables en Matemáticas y en programación

En Matemáticas, una variable representa una cantidad, conocida o desconocida, dentro de una expresión. Por ejemplo, en `x + 3 = 5`, se puede averiguar que `x` vale 2. En una fórmula como `y = x + 1`, en cambio, se pueden probar distintos valores de `x` y calcular el valor correspondiente de `y`.

En programación, una variable es un nombre al que se asigna un valor para poder utilizarlo más adelante. El signo `=` indica una **asignación**, no una igualdad matemática: Python calcula lo que aparece a la derecha y asigna el resultado al nombre de la izquierda. Por eso `total = precio * unidades` se lee como «calcula el producto y guarda el resultado en `total`», no como una ecuación que haya que resolver.

5.1 Asignación

Para crear una variable, escribe su nombre, el signo `=` y el valor que quieres asignarle:

```
nombre = "Ana"
edad = 28
```

La asignación puede guardar un valor escrito directamente o el resultado de un cálculo. Python evalúa primero lo que aparece a la derecha y luego guarda ese resultado con el nombre de la izquierda: En el intérprete interactivo, escribir el nombre muestra su valor. En un archivo `.py`, una línea que solo contiene un nombre no lo muestra en pantalla; para ello se utiliza `print()`. Esta diferencia importa al probar instrucciones en la consola y al ejecutar un programa guardado en un archivo.

```
horas = 2
minutos = horas * 60
print(minutos)  # 120
```

En el intérprete interactivo, escribir el nombre muestra su valor. En un archivo `.py`, para mostrarlo en pantalla se utiliza `print()`.

```
>>> edad = 28
>>> edad
28
```

El texto se escribe entre comillas simples o dobles. Sin comillas, Python interpreta la palabra como el nombre de otra variable:

```
>>> nombre = "Pepito Conejo"
>>> nombre
'Pepito Conejo'
>>> nombre = Pepe
Traceback (most recent call last):
```

```
    ...
NameError: name 'Pepe' is not defined
```

Las comillas marcan dónde empieza y termina el texto. En cambio, sin comillas, `Pepe` no es texto: Python intenta encontrar una variable llamada `Pepe`.

5.2 Reasignación

Se puede asignar un valor nuevo a una variable. La asignación nueva reemplaza el valor anterior:

```
edad = 28
edad = 29
print(edad)  # 29
```

También se puede usar el valor actual para calcular el siguiente:

```
puntuacion = 10
puntuacion = puntuacion + 5
print(puntuacion)  # 15
```

La expresión de la derecha se calcula primero. Por eso `puntuacion = puntuacion + 5` significa «toma el valor actual, súmale 5 y asigna el resultado otra vez a `puntuacion`». Esta forma se utiliza para llevar una cuenta acumulada, actualizar una puntuación o aumentar un importe. El nombre debe tener un valor antes de poder aparecer en la expresión de la derecha.

5.3 Intercambiar valores

Para intercambiar los valores de dos variables, se puede utilizar una tercera variable temporal:

```
primero = 5
segundo = 10

temporal = primero
primero = segundo
segundo = temporal

print(primero)  # 10
print(segundo)  # 5
```

La variable `temporal` evita perder uno de los valores. Si se escribiera primero `primero = segundo`, el valor inicial de `primero` se reemplazaría; al hacer después `segundo = primero`, ambas variables acabarían valiendo lo mismo. La variable temporal conserva el primer valor mientras se realizan las otras asignaciones. Para separar palabras, se recomienda usar guiones bajos, por ejemplo `fecha_de_nacimiento`. Esta forma se llama `snake_case` y facilita la lectura. También se pueden usar mayúsculas dentro del nombre, pero Python distingue entre ellas: `precio` y `Precio` no son el mismo nombre.

5.4 Nombres de variables

Elige nombres que describan el valor, como `precio` o `numero_de_intentos`. Las reglas básicas son:

- El nombre puede contener letras, números y guiones bajos (_).
- Debe empezar por una letra o un guion bajo, no por un número.
- No puede contener espacios.
- Python distingue mayúsculas de minúsculas: `nombre` y `Nombre` son nombres distintos.
- No se pueden usar palabras reservadas como `for` o `if`.

Para separar palabras, se recomienda usar guiones bajos, por ejemplo `fecha_de_nacimiento`. Esta forma se llama `snake_case`.

```
nombre_completo = "Lucía García"
numero_de_intentos = 3
```

Conviene evitar nombres demasiado cortos o que no expliquen el dato. Por ejemplo, `p` puede ser difícil de entender, mientras que `precio` indica claramente qué representa. El nombre no cambia el valor ni el resultado del cálculo: solo ayuda a quien lee el programa.

5.5 Asignaciones aumentadas

Las asignaciones aumentadas actualizan una variable a partir de su valor actual. Se escriben con un operador aritmético seguido de `=`:

```
contador = 0
contador += 1  # equivale a contador = contador + 1
contador *= 2  # equivale a contador = contador * 2
```

En la primera línea, `contador += 1` suma uno al valor actual. En la segunda, `contador *= 2` multiplica ese nuevo valor por dos. También existen `-=`, `/=`, `//=`, `%=`, y `**=` para restar, dividir, calcular una división entera, obtener el resto y elevar a una potencia.

Python no tiene operadores `++` ni `--`; para aumentar o reducir una unidad se usa `+= 1` o `-= 1`. La variable debe haberse definido antes de aplicar una asignación aumentada.

5.6 Constantes

Python no tiene una instrucción que impida modificar una variable. Para indicar que un valor no debería cambiar, se escribe su nombre en mayúsculas y se evita reasignarlo:

```
MAXIMO_INTENTOS = 3
```

Las mayúsculas son una convención, no una protección: Python permite cambiar el valor con otra asignación. No existe una forma incorporada de declarar una constante que el propio lenguaje impida modificar.

Se suelen usar constantes para valores que tienen un significado fijo dentro del programa, como un límite o un porcentaje. Darles un nombre evita repetir un número sin explicar qué representa:

```
IVA = 0.21
precio_final = 100 * (1 + IVA)
```

Por convención, el nombre de una constante se escribe en mayúsculas y con guiones bajos entre palabras. La persona que programa debe respetar esa convención y no reasignarle otro valor.

5.7 Borrar una variable

La instrucción `del` elimina un nombre. Si se intenta utilizar después, Python produce un `NameError`:

```
nombre = "Ana"
del nombre
# print(nombre)  # NameError: el nombre ya no existe
```

`del` no es necesario para cambiar el valor: para eso basta con hacer otra asignación. Se puede usar cuando se quiere dejar de utilizar un nombre. Después de borrarlo, hay que asignarle un valor de nuevo antes de volver a consultarlo.

5.8 Error frecuente: usar un nombre antes de asignarlo

Una variable debe recibir un valor antes de utilizarse:

```
# print(ciudad)  # NameError: ciudad aún no está definida
ciudad = "Sevilla"
print(ciudad)
```

Un `NameError` también puede deberse a un error al escribir el nombre. Python distingue mayúsculas y minúsculas, así que `ciudad`, `Ciudad` y `CIUDAD` se interpretan como nombres diferentes. Para localizar el problema, compara el nombre de la asignación con el de la instrucción que lo utiliza.

Actividades 5

Actividad 1: ficha de variables

Crea un programa que asigne un nombre y una edad a dos variables. Muestra ambos valores. Después cambia la edad y vuelve a mostrarla.

Actividad 2: reasignar variables

Asigna `10` a `a` y luego asigna el valor de `a` a `b`. Muestra ambas variables. Cambia `a` a `20` y vuelve a mostrar las dos. Explica por qué `b` sigue valiendo `10`.

Actividad 3: calcular y actualizar un valor

Asigna un precio y una cantidad a dos variables. Calcula el importe en una tercera variable y muéstralo. Aumenta la cantidad con `+=` y calcula de nuevo el importe.

Actividad 4: elegir nombres válidos

Indica cuáles de estos nombres se pueden usar como variables y explica por qué: `2precio`, `precio total`, `for`, `precio_total` y `precioUnitario`. Después elige nombres descriptivos para guardar el precio y la cantidad de un producto.

Actividad 5: conversor de distancia

Asigna una distancia en kilómetros a una variable y calcula su equivalencia en metros en otra variable. Muestra ambos valores con `print()`. Después modifica la distancia en kilómetros y vuelve a calcular la equivalencia en metros.

Actividad 6: constante por convención

Define `MAXIMO_INTENTOS` con el valor `3`. Explica por qué se escribe en mayúsculas y qué ocurriría si después le asignas el valor `5`. ¿Python impide el cambio o depende de la persona que escribe el programa respetar la convención?

Actividad 7: seguir las asignaciones

Sin ejecutar el código, completa el valor de `total` después de cada instrucción. Después comprueba tus respuestas ejecutándolo:

```
total = 12
total = total + 5
descuento = 3
total = total - descuento
```

Instrucción ejecutada	Valor de `total`
`total = 12`	_____
`total = total + 5`	_____
`total = total - descuento`	_____

Explica por qué la última instrucción utiliza el valor actual de `total`.

Actividad 8: corregir nombres

En cada línea hay un problema con el nombre utilizado. Explica el error y reescribe la instrucción correctamente:

```
2precio = 4.5
nombre completo = "Ana"
for = 3
ciudad = "Cádiz"
print(Ciudad)
```

En la última línea, utiliza el nombre que se definió en la instrucción anterior.

Actividad 9: actualizar cantidades

Empieza con `saldo = 50`. Aplica, en orden, estas operaciones usando asignaciones aumentadas: ingresa 20, gasta 15 y duplica el saldo restante. Escribe el saldo después de cada operación y comprueba el resultado con `print()`.

Actividad 10: preparar un recibo

Define variables para el nombre de un producto, su precio y la cantidad comprada. Calcula el subtotal y el total con IVA, usando una constante llamada `IVA`. Muestra el nombre del producto y ambos importes. Después cambia la cantidad y vuelve a calcular los importes.

---

6. Tipos de datos

⬆ Volver al índice

El tipo de dato indica qué clase de valor representa un dato y qué operaciones tienen sentido para él. En este apartado se explican los tipos que se utilizan en los ejemplos: valores lógicos, números, texto y varias formas de agrupar valores.

6.1 Booleanos: `bool`

Un booleano representa un resultado que solo puede ser `True` (verdadero) o `False` (falso). Las comparaciones producen valores booleanos. Se pueden guardar en variables y utilizar en condiciones para decidir qué instrucciones ejecutar.

En este ejemplo, `18 >= 18` es verdadero, por lo que `es_mayor` recibe `True`. `type()` permite comprobar el tipo del valor guardado:

```
es_mayor = 18 >= 18
print(es_mayor)       # True
print(type(es_mayor)) # <class 'bool'>
```

Así, `bool` sirve para expresar el resultado de una pregunta que admite una respuesta verdadera o falsa, como comprobar si se cumple una comparación.

6.2 Enteros: `int`

El tipo `int` representa números sin parte decimal; puede ser positivo, negativo o cero. Los ejemplos muestran distintas maneras de escribirlos: `150` y `-3` están en decimal, mientras que `0b`, `0o` y `0x` indican, respectivamente, que el número está escrito en binario, octal o hexadecimal. La base cambia la escritura, no el hecho de que el valor sea un entero. Por ejemplo, `0b1010` representa diez y tanto `0o52` como `0x2A` representan cuarenta y dos.

```
usuarios = 150
temperatura = -3
binario = 0b1010
octal = 0o52
hexadecimal = 0x2A
```

6.3 Reales: `float`

El tipo `float` representa números que pueden tener parte decimal. En Python, la parte decimal se escribe después de un punto, como en `-15.75`. Internamente estos números se representan en binario, por lo que algunos valores decimales no se almacenan con exactitud y ciertos cálculos pueden mostrar una pequeña diferencia, como `0.1 + 0.2`.

Para controlar cuántas cifras decimales se muestran, se puede usar `:.2f` dentro de una cadena formateada. El `2` indica que se mostrarán dos cifras después del punto; se puede sustituir por otra cantidad. Este formato cambia cómo se presenta el número, no el valor almacenado.

```
saldo = -15.75
print(0.1 + 0.2)  # puede mostrar 0.30000000000000004

precio = 12.5
print(f"{precio:.2f}")  # 12.50
```

Debido a la representación binaria de los números de punto flotante, algunos resultados son aproximados. Para comparar importes decimales exactos, puede ser necesario utilizar el módulo `decimal`.

6.4 Cadenas: `str`

El tipo `str` representa texto como una secuencia de caracteres. Las comillas simples o dobles delimitan una cadena de una línea; las comillas triples permiten escribirla en varias líneas. Una cadena es inmutable: sus caracteres no se cambian directamente y los métodos que transforman el texto devuelven una cadena nueva.

```
nombre = "Ada Lovelace"
saludo = 'Hola'
texto_largo = """Primera línea
Segunda línea"""
```

Operaciones básicas:

```
texto = "Python"
print(len(texto))
print(texto[0])
print(texto[-1])
print(texto[0:3])
print(texto.lower())
print(texto.upper())
print("th" in texto)
```

En estas expresiones, `texto` vale `"Python"`:

- `len(texto)` devuelve el número de caracteres: `6`.
- `texto[0]` obtiene el carácter de la primera posición (`"P"`). La numeración empieza en cero; `texto[-1]` obtiene el último carácter (`"n"`).
- `texto[0:3]` obtiene desde la posición 0 hasta antes de la 3: `"Pyt"`.
- `texto.lower()` devuelve una versión en minúsculas y `texto.upper()` una versión en mayúsculas.
- `"th" in texto` comprueba si el fragmento está en la cadena y devuelve `True` o `False`. `not in` comprueba si no está.
- `"Hola " + "Ada"` concatena dos cadenas; `"ja" * 3` repite una cadena tres veces.

Métodos básicos de `str`

Cada método se llama escribiendo un punto después de la cadena o de la variable. Los que generan texto devuelven otra cadena y no modifican la original.

Método	Qué hace	Ejemplo
`lower()`	Devuelve el texto en minúsculas.	`"HOLA".lower()` da `"hola"`.
`upper()`	Devuelve el texto en mayúsculas.	`"hola".upper()` da `"HOLA"`.
`capitalize()`	Pone en mayúscula el primer carácter y el resto en minúscula.	`"hola mundo".capitalize()` da `"Hola mundo"`.
`title()`	Pone en mayúscula la inicial de cada palabra.	`"hola mundo".title()` da `"Hola Mundo"`.
`strip()`	Quita espacios en blanco del principio y del final.	`" hola ".strip()` da `"hola"`.
`replace(viejo, nuevo)`	Sustituye las apariciones de un texto por otro.	`"casa".replace("a", "o")` da `"coso"`.
`split(separador)`	Divide la cadena por el separador y devuelve sus partes en una lista.	`"rojo,azul".split(",")` da `["rojo", "azul"]`.
`join(partes)`	Une cadenas poniendo el texto que llama al método entre ellas.	`", ".join(["rojo", "azul"])` da `"rojo, azul"`.
`find(texto)`	Devuelve la posición de la primera aparición; devuelve `-1` si no lo	`"Python".find("th")` da `2`.

Método	Qué hace	Ejemplo
	encuentra.	
`count(texto)`	Cuenta cuántas veces aparece el texto indicado.	`"banana".count("a")` da `3`.
`startswith(texto)`	Comprueba si la cadena empieza con el texto indicado.	`"Python".startswith("Py")` da `True`.
`endswith(texto)`	Comprueba si la cadena termina con el texto indicado.	`"Python".endswith("on")` da `True`.
`format(valores)`	Inserta valores en los huecos `{}` de una cadena.	`"Hola, {}".format("Ada")` da `"Hola, Ada"`.

6.5 Listas: `list`

Una lista (`list`) reúne varios elementos en un orden concreto y permite cambiar su contenido. En el ejemplo, `append("kiwi")` añade un elemento al final; `frutas[0] = "plátano"` reemplaza el de la primera posición. La primera posición tiene índice `0`. Las listas pueden contener valores de distintos tipos, aunque suele ser más claro agrupar valores que cumplen una misma función.

```
frutas = ["manzana", "pera", "uva"]
frutas.append("kiwi")
frutas[0] = "plátano"
print(frutas)
```

Métodos más comunes de las listas

Estos métodos se llaman con un punto después de la variable, por ejemplo `frutas.append("kiwi")`. Los métodos que modifican la lista actúan sobre la lista original; no hay que guardar su resultado en otra variable.

Método	Qué hace	Ejemplo
`append(valor)`	Añade un elemento al final.	`frutas.append("kiwi")`
`extend(valores)`	Añade al final todos los elementos de otra secuencia.	`numeros.extend([4, 5])`
`insert(indice, valor)`	Inserta un elemento en la posición indicada.	`frutas.insert(0, "kiwi")`
`remove(valor)`	Elimina la primera aparición del valor indicado.	`frutas.remove("pera")`
`pop(indice)`	Elimina y devuelve el elemento de la posición indicada; si se omite, quita el último.	`ultimo = frutas.pop()`
`clear()`	Elimina todos los elementos de la lista.	`frutas.clear()`
`index(valor)`	Devuelve la posición de la primera aparición del valor.	`frutas.index("pera")`

Método	Qué hace	Ejemplo
`count(valor)`	Cuenta cuántas veces aparece el valor.	`frutas.count("pera")`
`sort()`	Ordena la lista original de menor a mayor.	`numeros.sort()`
`reverse()`	Invierte el orden de los elementos de la lista original.	`frutas.reverse()`
`copy()`	Crea una copia de la lista.	`copia = frutas.copy()`

Las funciones `len()` y `sum()` también se pueden usar con listas, aunque no son métodos: `len(numeros)` cuenta los elementos y `sum(numeros)` suma sus valores.

```
numeros = [4, 1, 8, 2]
numeros.sort()
print(numeros)       # [1, 2, 4, 8]
print(len(numeros))  # 4
print(sum(numeros))  # 15
```

6.6 Array: `array.array`

`array.array` es el tipo de array que proporciona el módulo estándar `array`. Se parece a una lista porque mantiene los valores ordenados y permite modificarlos, pero todos sus elementos deben ser del mismo tipo numérico. Al crearlo se indica el tipo mediante un código: `"i"` para enteros y `"f"` para números de punto flotante. Esta restricción permite almacenar secuencias numéricas de forma compacta.

¿Cuándo usar una lista y cuándo un array?

En Python, la **lista suele ser la opción habitual**: sirve para la mayoría de colecciones y permite guardar valores de distintos tipos. `array.array` se usa en un caso más concreto: cuando todos los elementos son numéricos del mismo tipo y se quiere almacenarlos de forma compacta.

Necesidad	Opción adecuada	Motivo
Guardar una colección general, que puede mezclar tipos	`list`	Es flexible y no requiere indicar de antemano el tipo de sus elementos.
Guardar una secuencia compacta de números todos del mismo tipo	`array.array`	El código elegido al crearlo fija el tipo permitido para sus elementos.

Por ejemplo, para una lista de edades normalmente basta con una lista. Si se necesita una secuencia compacta de números enteros, se puede elegir `array("i", ...)`:

```
from array import array

edades_lista = [18, 21, 25]
edades_array = array("i", [18, 21, 25])
```

```
edades_array.append(30)  # añade un entero; debe coincidir con el código "i"
edades_array[0] = 19     # cambia el valor de la primera posición
print(edades_lista)
print(edades_array)

temperaturas = array("f", [18.5, 21.0, 19.5])
```

En ambos casos se puede consultar un elemento por su posición, añadir valores y cambiar un elemento. La diferencia no es que uno sustituya al otro: usa una lista como opción general y elige `array.array` cuando la restricción a un único tipo numérico y el almacenamiento compacto sean útiles. La documentación oficial del módulo `array` contiene los códigos disponibles.

6.7 Tuplas: `tuple`

Una tupla (`tuple`) agrupa valores en un orden concreto, como una lista, pero no permite cambiar sus elementos una vez creada. En el ejemplo, `punto` contiene dos valores y la asignación `x, y = punto` guarda el primero en `x` y el segundo en `y`.

```
punto = (10, 20)
x, y = punto
```

La coma es la que indica que `(7,)` es una tupla de un único elemento. Sin ella, `(7)` solo agrupa el número entre paréntesis:

```
un_elemento = (7,)
```

6.8 Conjuntos: `set`

Un conjunto (`set`) agrupa elementos sin repetirlos y no los organiza por posiciones. En el ejemplo, `"rojo"` aparece una sola vez aunque se haya escrito dos veces. El operador `&` obtiene los elementos que están en ambos conjuntos; `|` reúne los elementos de los dos. Por eso su resultado común contiene `"editor"`, y el conjunto combinado contiene `"admin"`, `"editor"` e `"invitado"`.

```
colores = {"rojo", "verde", "rojo"}
print(colores)  # solo contiene un "rojo"

permitidos = {"admin", "editor"}
solicitados = {"editor", "invitado"}
print(permitidos & solicitados)
print(permitidos | solicitados)
```

Para crear un conjunto vacío se usa `set()`. Las llaves vacías `{}` representan un diccionario vacío, no un conjunto.

6.9 Diccionarios: `dict`

Un diccionario (`dict`) guarda asociaciones entre claves y valores. En el ejemplo, `"nombre"` permite localizar `"Elena"`, `"edad"` permite localizar `31` y `"activo"` permite localizar `True`. Para consultar un valor se escribe su clave entre corchetes, como en `persona["nombre"]`:

```
persona = {
    "nombre": "Elena",
    "edad": 31,
    "activo": True,
}
print(persona["nombre"])  # Elena
```

La clave también permite cambiar el valor asociado o añadir una asociación nueva:

```
persona["edad"] = 32
persona["ciudad"] = "Cádiz"
```

```
saludo = "hola"
saludo_mayusculas = saludo.upper()
print(saludo)             # hola
print(saludo_mayusculas)  # HOLA
```

`upper()` devuelve una cadena nueva y no modifica `saludo`; por eso la primera impresión sigue mostrando `"hola"`. En cambio, `sort()` modifica la lista sobre la que se llama y devuelve `None`. La función `sorted()` ordena los elementos y devuelve una lista nueva, sin cambiar la lista original:

```
numeros = [3, 1, 2]
resultado = numeros.sort()
print(numeros)   # [1, 2, 3]
print(resultado) # None

numeros = [3, 1, 2]
ordenados = sorted(numeros)
print(numeros)   # [3, 1, 2]
print(ordenados) # [1, 2, 3]
```

```
if not nombre:
    print("El nombre está vacío")
```

---

7. Literales

⬆ Volver al índice

Un literal es una notación escrita directamente en el código para representar un valor. No es lo mismo un literal que una variable: `25` es un literal entero, mientras que `edad` es un nombre que puede referirse a un objeto entero.

7.1 Literales numéricos

```
entero = 100
negativo = -25
real = 2.5
notacion_cientifica = 1.2e3
binario = 0b1101
```

Los guiones bajos pueden separar grupos de cifras para que los números largos sean más fáciles de leer. Python los ignora al interpretar el literal: `1_000_000` y `1000000` representan exactamente el mismo valor. No son un operador ni cambian el número.

```
presupuesto = 1_000_000
presupuesto_sin_separadores = 1000000
print(presupuesto == presupuesto_sin_separadores)  # True
```

7.2 Literales booleanos y nulo

```
disponible = True
eliminado = False
sin_resultado = None
```

7.3 Literales de cadena

Las comillas delimitan el texto y le indican a Python dónde empieza y termina la cadena. Se pueden usar comillas simples (`'...'`) o dobles (`"..."`): ambas crean una cadena y tienen el mismo significado. A menudo se elige el tipo de comilla que permita escribir el texto con menos escapes:

```
mensaje1 = "Ella dijo 'hola'"   # comillas dobles por fuera
mensaje2 = 'Ella dijo "hola"'   # comillas simples por fuera
mensaje3 = "It's Python"        # el apóstrofo no necesita escape
```

Si el texto contiene la misma comilla que delimita la cadena, se puede escribir una barra invertida (`\`) antes de esa comilla para incluirla. La barra invertida también introduce secuencias de escape: combinaciones que representan caracteres especiales.

```
comillas = "Ella dijo \"hola\""
apostrofo = 'It\'s Python'
multilinea = "Primera línea\nSegunda línea"
tabulacion = "Nombre:\tAda"
ruta_con_barra = "C:\\Users\\Ana"
```

- `\n` representa un salto de línea.
- `\t` representa una tabulación.
- `\\` representa una barra invertida dentro de la cadena.
- `\"` y `\'` permiten incluir comillas dobles o simples, respectivamente.

Si se antepone `r` a una cadena, como en `r"C:\Users\Ana\archivo.txt"`, se crea una cadena cruda: las barras invertidas se conservan y no se interpretan como el inicio de estas secuencias. Las comillas triples permiten escribir cadenas que ocupan varias líneas:

```
multilinea = """Primera línea
Segunda línea"""
```

Las f-strings permiten insertar valores o expresiones entre llaves:

```
producto = "libro"
precio = 19.9
print(f"{producto}: {precio:.2f} €")
```

---

8. Constantes

⬆ Volver al índice

Python no impone constantes inmutables mediante una palabra reservada. La convención consiste en escribir en mayúsculas los nombres cuyo valor no debería cambiar durante la ejecución:

```
PI = 3.141592653589793
MAX_INTENTOS = 3
NOMBRE_APLICACION = "Gestor de tareas"
```

La convención comunica una intención, pero no impide la reasignación:

```
MAX_INTENTOS = 3
MAX_INTENTOS = 10  # Python lo permite, aunque puede ser un error de diseño
```

8.1 Constantes agrupadas en un módulo

Archivo `configuracion.py`:

```
TASA_IVA = 0.21
MONEDA = "EUR"
```

Otro archivo puede importarlas:

```
from configuracion import TASA_IVA

precio_final = 100 * (1 + TASA_IVA)
```

8.2 `Final` para expresar intención estática

`Final` se importa desde `typing` y se escribe como anotación junto a una variable para indicar que su valor no debería reasignarse. «Estática» significa que esta intención puede ser revisada por el editor o por una herramienta de análisis sin ejecutar el programa.

Si una herramienta detecta que se asigna otro valor a `PI`, puede mostrar un aviso. Sin embargo, `Final` no bloquea la asignación mientras el programa se ejecuta: Python permite reasignar la variable. Por tanto, sirve para comunicar la intención y ayudar a detectar errores, pero no convierte el nombre en una constante protegida.

```
from typing import Final

PI: Final = 3.141592653589793
# PI = 3.14  # Una herramienta de análisis puede señalar esta reasignación.
```

8.3 Cuándo usar constantes

Una constante es apropiada para valores que:

- Se repiten en varios lugares.
- Tienen un significado de negocio o configuración.
- Podrían cambiar en el futuro en un único punto.
- No dependen de los datos de una ejecución concreta.

Es mejor escribir `SEGUNDOS_POR_MINUTO = 60` que repetir el número `60` sin explicar su significado.

Actividades 8

Desarrolla y ejecuta un programa para cada ejercicio.

Actividad 1: ficha de matrícula

Prepara una ficha para una alumna llamada Lucía Ruiz. Incluye su edad (`19`), nota media (`8.4`), si está matriculada (`True`) y su fecha de baja, que todavía se desconoce. Guarda cada dato por separado. Al ejecutar el programa, muestra primero todos los datos en una ficha legible y después el tipo de cada valor. El resultado debe distinguir el dato desconocido del texto `"None"`.

Actividad 2: cuenta institucional

Pide por separado el nombre y los apellidos. Prueba el programa con `mARía` y `lÓPEZ` . La salida debe mostrar el nombre como `María López` y crear la cuenta `maría.lópez@centro.example`, sin espacios sobrantes y en minúsculas. El programa debe aceptar otros nombres introducidos por teclado, no limitarse a los datos de prueba.

Actividad 3: informe de incidencias

Pide una descripción de una incidencia. Prueba el programa con `ERROR en aula. Error al arrancar` . Presenta cuatro datos: la descripción sin espacios sobrantes; una versión con las palabras en formato de título; una versión en minúsculas con los espacios reemplazados por guiones; y la cantidad de veces que aparece `error`, contando igual `ERROR` y `Error`. Calcula también la cantidad de caracteres de la descripción limpia. Presenta cada resultado con una etiqueta clara.

Actividad 4: selección de asignaturas

La entrada inicial es la cadena `" Python, Redes , Sistemas, Python "` (con espacios sobrantes al principio, al final y alrededor de las comas). Convierte sus elementos en una selección sin espacios sobrantes ni asignaturas repetidas. Añade `Bases de datos`, elimina `Redes` porque ya no se imparte y muestra la selección final en orden alfabético, separando las asignaturas con `|`. La línea final debe ser `Bases de datos | Python | Sistemas`.

Actividad 5: lista de compra compartida

La compra inicial es `pan`, `leche`, `manzanas` y `leche`. Añade `arroz` y `café`, elimina `manzanas` y ordena alfabéticamente lo que queda. Muestra la lista resultante y cuántas unidades de `leche` hay. El resultado final debe conservar las dos entradas de leche, ya que representan dos unidades compradas.

Actividad 6: organización de una playlist

La playlist inicial es `Luz`, `Viaje`, `Luz`, `Norte`. Añade `Casa` y `Sol`, y quita solo una de las dos canciones llamadas `Luz`. Muestra la playlist en su orden actual y, aparte, una versión alfabética. La versión alfabética no debe cambiar el orden de la playlist original. Indica cuántas veces aparece `Luz` y en qué posición está su primera aparición en la playlist actual.

Actividad 7: revisión de archivos entregados

Pide el nombre de un archivo y prepara una versión normalizada sin espacios exteriores y en minúsculas. Para esa versión, muestra si comienza con `act_`, si termina en `.py` y la posición del primer guion bajo. Comprueba el programa con `ACT_Cadenas.PY` y `notas-final.PY`. Para el primer nombre debe indicar que cumple ambos formatos y que el guion bajo está en la posición `3`; para el segundo, que no empieza por `act_`, que sí termina en `.py` y que la búsqueda del guion bajo da `-1`.

Actividad 8: registro de temperaturas

Registra las temperaturas enteras `18`, `20`, `21`, `19` y `22` de cinco aulas. Después llega la lectura de una sexta aula, `23`, y se descubre que la tercera lectura debía ser `20`. Actualiza el registro y muestra los seis valores en el orden de las aulas, junto con el tipo de dato de la estructura elegida. La secuencia final debe contener `18`, `20`, `20`, `19`, `22` y `23`, y todos sus elementos deben seguir siendo enteros.

Actividad 9: configuración de una aplicación

Prepara la configuración de una aplicación llamada `Aula ASIR`, versión `1.2`, idioma `es` y límite de `3` intentos. Guarda estos valores para que se puedan cambiar desde un único lugar y nómbralos como constantes según la convención explicada en el tema. Marca la versión con `Final`. Muestra una pantalla de bienvenida que incluya el nombre, la versión, el idioma y el límite de intentos. No hace falta crear otros archivos.

Actividad 10: catálogo de productos

El catálogo `Equipamiento de aula` empieza con estos nombres: `teclado`, `RATÓN`, `monitor` y `Ratón`. Pide dos productos nuevos: prueba con `ALTAVOCES` y `webcam` . Limpia los espacios exteriores y unifica las mayúsculas para que nombres iguales se reconozcan como el mismo producto. Retira `monitor`, conserva una sola entrada de cada producto y muestra el resultado en orden alfabético, separado por comas. Guarda el nombre del catálogo como constante y muestra también el número de productos finales. La lista mostrada debe contener `Altavoces`, `Ratón`, `Teclado` y `Webcam`, una vez cada uno; el total es `4`.

---

9. Operadores y expresiones

⬆ Volver al índice

Una expresión combina valores, variables, operadores y llamadas para producir un resultado.

9.1 Tipos numéricos y operadores aritméticos

Python trabaja principalmente con enteros (`int`), números decimales (`float`) y números complejos (`complex`). En los literales decimales se usa un punto, no una coma: `3.5` es un número decimal, mientras que `3,5` crea una pareja de valores.

Operación	Qué hace	Ejemplo
`+`	Suma dos valores.	`print(17 + 5) # 22`
`-`	Resta el segundo valor al primero.	`print(17 - 5) # 12`
`*`	Multiplica dos valores.	`print(17 * 5) # 85`
`/`	Divide y devuelve un resultado decimal.	`print(17 / 5) # 3.4`
`//`	Divide y redondea el cociente hacia abajo.	`print(17 // 5) # 3`
`%`	Devuelve el resto de la división.	`print(17 % 5) # 2`

Operación	Qué hace	Ejemplo
`**`	Eleva un número a una potencia.	`print(2 ** 3) # 8`
`round()`	Redondea un número a las cifras indicadas.	`print(round(4.3527, 2)) # 4.35`
`abs()`	Devuelve el valor absoluto.	`print(abs(-7)) # 7`
`max()` / `min()`	Devuelven el mayor o el menor de los valores.	`print(max(4, 8, 2)) # 8` y `print(min(4, 8, 2)) # 2`
`sum()`	Suma los elementos de un iterable numérico.	`print(sum([1, 2, 3])) # 6`

Los guiones bajos permiten mejorar la lectura de números largos sin cambiar su valor:

```
poblacion = 48_000_000
micras = 0.000_001
```

También se pueden escribir números en binario (`0b`), octal (`0o`) y hexadecimal (`0x`):

```
binario = 0b1010       # 10
octal = 0o12           # 10
hexadecimal = 0xA      # 10
```

Operaciones básicas

```
a = 17
b = 5

print(a + b)   # suma: 22
print(a - b)   # resta: 12
print(a * b)   # multiplicación: 85
print(a / b)   # división real: 3.4
print(a // b)  # división entera: 3
print(a % b)   # resto: 2
print(a ** b)  # potencia: 1419857
```

`/` siempre produce una división real. `//` redondea hacia abajo, lo que conviene tener en cuenta con números negativos:

```
print(-7 // 3)  # -3
```

La división entera `//` devuelve el cociente redondeado hacia abajo, no el cociente truncado hacia cero. El resto `%` y el cociente están relacionados por la expresión `a == (a // b) * b + (a % b)`.

```
print(11 // 3)       # 3
print(11 % 3)        # 2
print(divmod(11, 3)) # (3, 2)
```

`divmod()` devuelve en una tupla el cociente y el resto. Tanto `/` como `//`, `%` y `divmod()` producen un error `ZeroDivisionError` si el divisor es cero.

Potencias y raíces

El operador `**` calcula potencias. Los exponentes negativos producen el inverso y los exponentes fraccionarios permiten calcular raíces:

```
print(2 ** 3)       # 8
print(10 ** -2)     # 0.01
print(9 ** 0.5)     # 3.0
print(pow(2, 3, 5)) # 3: (2 ** 3) % 5
```

Hay que usar paréntesis cuando el signo negativo forma parte de la base:

```
print(-2 ** 2)      # -4
print((-2) ** 2)    # 4
```

Redondeo y precisión

`round()` redondea un número. Su segundo argumento indica cuántas cifras decimales conservar y también puede ser negativo para redondear decenas, centenas, etc. En los casos exactamente intermedios, Python utiliza el redondeo al par más cercano.

```
print(round(4.3527))      # 4
print(round(4.3527, 2))   # 4.35
print(round(4352, -2))    # 4400
print(round(2.5))         # 2
print(round(3.5))         # 4
```

Los `float` se almacenan normalmente en formato binario y algunos decimales no se pueden representar exactamente. Por eso un cálculo sencillo puede mostrar una pequeña diferencia:

```
print(0.1 + 0.1 + 0.1)    # 0.30000000000000004
```

Para mostrar resultados al usuario se puede usar un formato como `:.2f`, pero no conviene redondear resultados intermedios que se reutilizarán en cálculos posteriores. Cuando se necesita exactitud decimal, se puede utilizar `decimal.Decimal`.

Funciones matemáticas habituales

Algunas operaciones frecuentes están disponibles como funciones integradas:

```
print(abs(-7))                  # 7
print(max(4, 8, 2))             # 8
print(min(4, 8, 2))             # 2
print(sum([1, 2, 3, 4]))         # 10
```

Para redondear hacia abajo o hacia arriba se puede importar el módulo `math`:

```
import math

print(math.floor(2.9))           # 2
print(math.ceil(2.1))            # 3
print(math.sqrt(25))             # 5.0
```

La información de esta ampliación se basa en Números y operaciones aritméticas elementales, de mclibre.org.

9.2 Operadores de comparación

Devuelven `True` o `False`:

Operación	Qué hace	Ejemplo
`==`	Comprueba si dos valores son iguales.	`print(20 == 20) # True`
`!=`	Comprueba si dos valores son distintos.	`print(20 != 18) # True`
`>`	Comprueba si el valor de la izquierda es mayor.	`print(20 > 18) # True`
`>=`	Comprueba si el valor de la izquierda es mayor o igual.	`print(20 >= 20) # True`
`<`	Comprueba si el valor de la izquierda es menor.	`print(18 < 20) # True`
`<=`	Comprueba si el valor de la izquierda es menor o igual.	`print(20 <= 20) # True`

```
edad = 20
print(edad == 20)
print(edad != 18)
print(edad > 18)
print(edad >= 20)
print(edad < 30)
print(edad <= 20)
```

9.3 Operadores lógicos

Operación	Qué hace	Ejemplo
`and`	Devuelve el primer operando falso o el último si todos son verdaderos.	`print(True and False) # False`
`or`	Devuelve el primer operando verdadero o el último si todos son falsos.	`print(0 or "sí") # sí`
`not`	Invierte el valor lógico: `True` pasa a `False` y viceversa.	`print(not True) # False`

```
es_mayor = edad >= 18
tiene_entrada = True

print(es_mayor and tiene_entrada)
print(es_mayor or tiene_entrada)
print(not tiene_entrada)
```

`and` devuelve el primer operando falso o el último si todos son verdaderos. `or` devuelve el primer operando verdadero o el último si todos son falsos. Esta evaluación perezosa permite patrones como:

```
nombre = entrada.strip() if entrada else "Anónimo"
```

9.4 Operadores de pertenencia

Operación	Qué hace	Ejemplo
`in`	Comprueba si un elemento está presente en una colección o como clave de un diccionario.	`print("uva" in ["pera", "uva"]) # True`
`not in`	Comprueba si un elemento no está presente en una colección.	`print("manzana" not in ["pera", "uva"]) # True`

```
frutas = ["pera", "uva"]
print("uva" in frutas)
print("manzana" not in frutas)
```

En un diccionario, `in` comprueba claves:

```
persona = {"nombre": "Ana"}
print("nombre" in persona)
```

9.5 Operadores de identidad

Operación	Qué hace	Ejemplo
`is`	Comprueba si dos referencias apuntan al mismo objeto.	`valor = None; print(valor is None) # True`
`is not`	Comprueba si dos referencias apuntan a objetos distintos.	`valor = 5; print(valor is not None) # True`

```
valor = None
print(valor is None)
print(valor is not None)
```

Para valores ordinarios se debe usar `==`, no `is`:

```
primero = [1, 2]
segundo = [1, 2]
print(primero == segundo)  # mismo contenido
print(primero is segundo)  # objetos distintos
```

9.6 Operadores de asignación aumentada

Operación	Qué hace	Ejemplo
`+=`	Suma el valor y guarda el resultado en la variable.	`contador = 0; contador += 1; print(contador) # 1`
`-=`	Resta el valor y guarda el resultado en la variable.	`contador = 2; contador -= 1; print(contador) # 1`
`*=`	Multiplica por el valor y guarda el resultado.	`contador = 3; contador *= 2; print(contador) # 6`
`/=`	Divide por el valor y guarda el resultado.	`total = 6; total /= 2; print(total) # 3.0`
`//=`	Divide hacia abajo y guarda el resultado.	`total = 7; total //= 2; print(total) # 3`
`%=`	Guarda el resto de la división.	`total = 7; total %= 2; print(total) # 1`
`**=`	Eleva a la potencia indicada y guarda el resultado.	`base = 3; base **= 2; print(base) # 9`
`&=` / `|=` / `^=`	Aplica AND, OR o XOR bit a bit y guarda el resultado.	
`<<=` / `>>=`	Desplaza los bits a la izquierda o a la derecha y guarda el resultado.	`bits = 0b1100; bits <<= 1; print(bits) # 24`

```
contador = 0
contador += 1
contador *= 2
contador -= 1
```

También existen `/=`, `//=`, `%=`, `**=`, `&=`, `|=`, `^=`, `<<=` y `>>=`.

9.7 Operadores bit a bit

Se aplican a enteros representados en binario:

Operación	Qué hace	Ejemplo
`&`	Pone a `1` los bits que son `1` en ambos operandos.	`print(0b1100 & 0b1010) # 8`
`` ` ``	`` ` ``	Pone a `1` los bits que son `1` en al menos un operando.
`^`	Pone a `1` los bits que son distintos entre sí.	`print(0b1100 ^ 0b1010) # 6`
`~`	Invierte los bits del entero, según la representación de enteros de Python.	`print(~0b1100) # -13`
`<<`	Desplaza los bits a la izquierda.	`print(0b1100 << 1) # 24`
`>>`	Desplaza los bits a la derecha.	`print(0b1100 >> 1) # 6`

```
a = 0b1100
b = 0b1010
print(a & b)
print(a | b)
print(a ^ b)
print(~a)
print(a << 1)
print(a >> 1)
```

Se usan en máscaras, permisos y programación de bajo nivel. No deben confundirse con `and` y `or`.

9.8 Precedencia y paréntesis

Python sigue una precedencia parecida a la matemática:

1. Paréntesis.
2. Potencias.
3. Signos unarios.
4. Multiplicación, división, división entera y resto.
5. Suma y resta.
6. Comparaciones.
7. `not`.

8. `and`.
9. `or`.

Operación	Qué hace	Ejemplo
Paréntesis	Se evalúan primero y permiten agrupar operaciones.	`print((2 + 3) * 4) # 20`
`**`	Se evalúa antes que los signos unarios y las operaciones aritméticas siguientes.	`print(2 ** 3 * 2) # 16`
Signos unarios (`+`, `-`, `~`)	Se aplican al valor; van antes que `*`, `/`, `//`, `%`, `+` y `-` binarios.	`print(-2 ** 2, (-2) ** 2) # -4 4`
`*`, `/`, `//`, `%`	Se evalúan antes que la suma y la resta.	`print(10 + 6 // 2) # 13`
`+`, `-`	Se evalúan antes que las comparaciones.	`print(2 + 3 == 5) # True`
Comparaciones	Se evalúan antes que `not`, `and` y `or`.	`print(3 < 5 and 5 < 8) # True`
`not`	Se evalúa antes que `and` y `or`.	`print(not False and True) # True`
`and`	Se evalúa antes que `or`.	`print(True or False and False) # True`
`or`	Se evalúa después de `and`.	`print(False or True) # True`

```
resultado = 2 + 3 * 4       # 14
resultado_claro = (2 + 3) * 4  # 20
```

Los paréntesis son recomendables cuando mejoran la intención, incluso si no son estrictamente necesarios.

9.9 Expresiones condicionales

Operación	Qué hace	Ejemplo
`valor_si_verdadero if condición else valor_si_falso`	Elige un valor según el resultado de la condición.	`edad = 17; tipo = "adulto" if edad >= 18 else "menor"; print(tipo) # menor`

```
edad = 17
tipo = "adulto" if edad >= 18 else "menor"
```

Conviene reservarlas para condiciones simples. Una expresión condicional anidada suele ser menos legible que un `if` normal.

Actividades 9

Actividad 1: calcular una factura

Crea un programa que lea precio, unidades y porcentaje de descuento, calcule subtotal, descuento, IVA y total, y muestre cada importe con dos decimales. Añade una condición que indique si el pedido supera un umbral de envío gratuito.

Actividad 2: descomponer una duración

Guarda en una variable una duración total de `7384` segundos. Calcula cuántas horas completas, minutos restantes y segundos restantes contiene, usando división entera (`//`) y resto (`%`). Muestra los tres resultados; deben ser `2` horas, `3` minutos y `4` segundos.

Actividad 3: comprobar el acceso a un aula

Guarda si una persona está matriculada (`True`), si tiene autorización especial (`False`) y su edad (`17`). Puede acceder si está matriculada y tiene al menos `18` años, o si tiene autorización especial. Calcula y muestra el resultado como `True` o `False`. Cambia los valores para comprobar también un caso en el que sí pueda acceder.

Actividad 4: actualizar las plazas disponibles

Un aula tiene `12` plazas disponibles. Reserva `3` plazas y después cancela `1` reserva, actualizando la variable con asignación aumentada. Muestra el número final de plazas y, mediante una expresión condicional, muestra `"Hay plazas"` si queda alguna o `"Completo"` si no queda ninguna.

---

10. Conversiones de tipo

⬆ Volver al índice

Cada valor de Python tiene un tipo, que determina qué operaciones se pueden realizar con él. Convertir es crear un valor de otro tipo a partir del valor original. Algunas conversiones son directas; otras no son posibles o pueden perder información.

La función `input()` siempre devuelve una cadena (`str`), aunque se escriban dígitos. Para hacer cálculos, hay que convertir esa cadena al tipo numérico adecuado:

```
edad_texto = input("Edad: ")  # Por ejemplo, "20"
edad = int(edad_texto)        # El entero 20
print(edad + 1)
```

10.1 Conversiones habituales

Estas funciones integradas crean valores de tipos habituales:

Operación	Qué hace	Ejemplo en Python
`int(valor)`	Convierte a entero si el valor tiene un formato aceptado.	`edad = int("20")`
`float(valor)`	Convierte a número decimal si el valor tiene un formato aceptado.	`precio = float("3.5")`
`str(valor)`	Obtiene una representación textual del valor.	`mensaje = str(2025)`
`bool(valor)`	Convierte el valor a `True` o `False` según si es verdadero o falso en contexto lógico.	`hay_datos = bool([1, 2])`
`list(iterable)`	Crea una lista con los elementos del iterable.	`letras = list("sol")`
`tuple(iterable)`	Crea una tupla con los elementos del iterable.	`coordenadas = tuple([3, 5])`
`set(iterable)`	Crea un conjunto, eliminando los elementos duplicados.	`unicos = set([1, 1, 2])`
`dict(iterable)`	Crea un diccionario a partir de pares clave-valor.	`edades = dict([("Ana", 20)])`

```
print(int("42"))                    # 42
print(float("3.14"))                # 3.14
print(str(2025))                    # "2025"
print(bool(1))                      # True
print(list("abc"))                  # ["a", "b", "c"]
print(tuple([1, 2]))                # (1, 2)
print(set([1, 1, 2]))               # {1, 2}
print(dict([("nombre", "Ana")]))    # {"nombre": "Ana"}
```

Una conversión numérica desde texto solo funciona si el contenido tiene un formato válido. En los literales decimales de Python se usa el punto:

```
print(int("42"))       # Correcto
print(float("3.14"))   # Correcto
# int("cuarenta")      # ValueError
# float("3,14")        # ValueError
```

10.2 Conversión explícita frente a implícita

Una conversión **explícita** se escribe en el programa, por ejemplo con `int()` o `str()`. Python también realiza algunas conversiones **implícitas** cuando una operación las necesita. En una operación entre un entero y un decimal, el entero se convierte a `float`:

```
resultado = 5 + 2.5
print(resultado)        # 7.5
print(type(resultado))  # <class 'float'>
```

Python no convierte automáticamente un texto en número ni un número en texto para concatenarlos. Hay que convertirlo o utilizar una cadena formateada:

```
edad = 25
print("Edad: " + str(edad))  # Conversión explícita a str
print(f"Edad: {edad}")       # Inserta el valor en una cadena formateada
# print("Edad: " + edad)     # TypeError: str y int no se concatenan
```

Conviene elegir el tipo según el uso: `int` para cantidades enteras, `float` para cálculos decimales y `str` para texto que se muestra o se procesa como caracteres.

10.3 Validar antes de convertir

Una cadena que no representa un número válido provoca `ValueError` al convertirla. Se puede validar el texto antes, pero las comprobaciones como `isdigit()` tienen límites: por ejemplo, no reconocen un signo `-` como parte del número ni permiten decimales. Para una entrada general, lo habitual es intentar la conversión y controlar el error:

```
texto = input("Introduce un entero: ").strip()
try:
    numero = int(texto)
except ValueError:
    print("El valor no es un entero válido")
else:
    print(numero * 2)
```

Operación	Qué hace	Ejemplo en Python
`texto.strip()`	Elimina espacios al principio y al final antes de convertir.	`numero = int(" 42 ".strip())`
`int(texto)`	Convierte el texto a entero; puede producir `ValueError`.	`numero = int("-8")`
`float(texto)`	Convierte el texto a decimal; puede producir `ValueError`.	`precio = float("3.5")`
`try` / `except ValueError`	Ejecuta una conversión y responde si el formato no es válido.	`try: numero = int(texto)`

Captura el error específico de conversión (`ValueError`) en lugar de ocultar cualquier error del programa. Si el usuario debe volver a intentarlo, se puede repetir la petición con un bucle:

```
while True:
    texto = input("Introduce un entero: ").strip()
    try:
        numero = int(texto)
    except ValueError:
        print("Escribe un número entero, por ejemplo -3 o 12.")
    else:
        break

print(f"El doble es {numero * 2}")
```

10.4 Conversiones con pérdida

Algunas conversiones no conservan exactamente el valor original. `int()` al recibir un `float` descarta la parte decimal, acercándose a cero; no redondea:

```
print(int(3.99))   # 3
print(int(-3.99))  # -3
```

Si se necesita redondear, se usa `round()`. En los casos exactamente intermedios, Python redondea al entero par más cercano:

```
print(round(3.99))       # 4
print(round(3.14159, 2)) # 3.14
print(round(2.5))        # 2
print(round(3.5))        # 4
```

Operación	Qué hace	Ejemplo en Python
`int(decimal)`	Elimina la parte fraccionaria y aproxima hacia cero.	`print(int(-3.99)) # -3`
`round(numero)`	Redondea al entero más cercano; en empates, al par más cercano.	`print(round(2.5)) # 2`
`round(numero, cifras)`	Redondea conservando las cifras decimales indicadas.	`print(round(3.14159, 2)) # 3.14`

Los números `float` tienen precisión finita, por lo que ciertos decimales no se representan exactamente en memoria. El formato de salida permite controlar lo que se muestra, pero no cambia el valor almacenado:

```
precio = 0.1 + 0.2
print(precio)       # 0.30000000000000004
print(f"{precio:.2f}")  # 0.30
```

10.5 Conversiones de estructuras

Las conversiones entre colecciones dependen del iterable de origen. Por ejemplo, convertir directamente un texto en lista separa sus caracteres; para separar palabras o campos se usa `split()`:

Operación	Qué hace	Ejemplo en Python
`list(texto)`	Crea una lista con un elemento por cada carácter.	`letras = list("sol")`
`texto.split(separador)`	Divide el texto en partes usando el separador.	`partes = "a,b".split(",")`
`tuple(iterable)`	Copia los elementos del iterable en una tupla.	`coordenadas = tuple([3, 5])`
`set(iterable)`	Crea un conjunto sin duplicados; no conserva un orden definido.	`colores = set(["rojo", "rojo", "azul"])`
`dict(pares)`	Construye un diccionario a partir de pares de dos elementos.	`datos = dict([("a", 1), ("b", 2)])`

```
texto = "rojo, verde, azul"
partes = texto.split(",")
colores = [elemento.strip() for elemento in partes]
print(colores)  # ["rojo", "verde", "azul"]

pares = [("a", 1), ("b", 2)]
diccionario = dict(pares)
print(diccionario)  # {"a": 1, "b": 2}
```

Al convertir a conjunto se eliminan los repetidos. Al convertir una lista a diccionario, cada elemento debe ser un par clave-valor válido; las claves repetidas se reemplazan por el último valor:

```
print(set([1, 1, 2]))  # {1, 2}
print(dict([("a", 1), ("a", 2)]))  # {"a": 2}
```

11. Comentarios

⬆ Volver al índice

Los comentarios son texto para las personas que leen el código. El intérprete los ignora. Sirven para explicar intención, decisiones o advertencias, no para repetir literalmente lo que hace cada línea.

11.1 Comentarios de una línea

```
# El descuento se aplica antes del IVA.
subtotal = precio * unidades
```

```
descuento = subtotal * porcentaje_descuento
```

Un comentario al final de línea debe ser corto:

```
limite = 100  # Máximo permitido por el negocio
```

11.2 Cadenas multilínea y docstrings

Las cadenas triples pueden utilizarse como docstrings cuando aparecen al principio de un módulo, clase o función:

```
def cuadrado(numero):
    """Devuelve el cuadrado de numero."""
    return numero ** 2
```

Una cadena triple colocada en cualquier lugar sin asignarla no es una buena forma de comentar. Para documentar elementos públicos se deben usar docstrings.

11.3 Qué comentar

Es útil comentar:

- Por qué se toma una decisión no evidente.
- Una limitación externa o regla de negocio.
- La razón de una solución aparentemente poco habitual.
- Un algoritmo complejo o una fórmula especializada.

No es útil escribir comentarios que contradigan el código o repitan nombres obvios:

```
# Incrementa contador en uno
contador += 1
```

11.4 Comentarios temporales

Los comentarios `TODO` pueden señalar trabajo pendiente:

```
# TODO: sustituir este almacenamiento temporal por una base de datos.
```

Deben revisarse antes de entregar el proyecto; un `TODO` olvidado puede indicar una funcionalidad incompleta.

---

12. Videotutoriales y recursos

⬆ Volver al índice

Para cada apartado se indican videotutoriales en español y enlaces a la documentación oficial de Python.

12.1 Introducción y objetivos

- Videotutoriales:
  - Curso Python 3 desde cero #1: Introducción e instalación (La Geekipedia De Ernesto)
  - Aprende Python: curso de Python desde cero (freeCodeCamp Español)
- Documentación: Tutorial de Python y Abriendo el apetito.

12.2 Entornos integrados de desarrollo

- Videotutoriales:
  - Instalación y configuración de Python en Visual Studio Code (Piogram)
  - Cómo configurar Visual Studio Code para Python (divcode)
  - ¿Qué son los entornos virtuales? (Enrique Barros)
  - Cómo crear un entorno virtual en Visual Studio Code (Código Espinoza)
- Documentación: Entornos virtuales y paquetes, Módulo `venv` y Python en VS Code.

12.3 Elementos de un programa

- Videotutoriales:
  - Palabras reservadas en Python (La Geekipedia De Ernesto)
  - Errores sintácticos en Python (Sergio A. Castaño Giraldo)
  - Tipos de errores en programación (Enrique Barros)
- Documentación: Análisis léxico, Palabras reservadas y Errores y excepciones.

12.4 Estructura y bloques fundamentales

- Videotutoriales:
  - Indentación y bloques en Python
  - Funciones, parámetros y return en Python
  - Módulos y if **name** == "**main**"
  - Alcance de variables y funciones en Python
- Documentación: Indentación, Definición de funciones, Módulos, `__main__` y Ámbitos y espacios de nombres.

12.5 Variables

- Videotutoriales:
  - Variables en Python explicadas fácil (PrograLabs)
  - Curso de Python #3: Variables y tipos de datos (Deividcoptero)
- Documentación: Python como calculadora y Sentencias de asignación.

12.6 Tipos de datos

- Videotutoriales:
  - Curso Python 3 desde cero #8: Tipos de datos (La Geekipedia De Ernesto)
  - Listas, tuplas, conjuntos y diccionarios en 8 minutos (BitBoss)

- Documentación: Tipos incorporados, Estructuras de datos y Módulo `array`.

12.7 Literales

- Videotutoriales:
  - Cadenas formateadas (f-strings) en Python (Jorge Escobar)
  - ¿Qué son los booleanos en Python? (Computando Código)
  - El tipo None en Python (Boot dev)
- Documentación: Literales.

12.8 Constantes

- Videotutoriales:
  - Constantes en Python (ConfiguroWeb)
  - Constantes y convenciones de nombres (Programa Con Arnau)
- Documentación: Constantes incorporadas y `typing.Final`.

12.9 Operadores y expresiones

- Videotutoriales:
  - Programación en Python: operadores aritméticos (Programación ATS)
  - Programación en Python: operadores lógicos (Programación ATS)
  - Operadores de comparación y lógicos (Enrique Barros)
- Documentación: Expresiones, Precedencia de operadores y Módulo `math`.

12.10 Conversiones de tipo

- Videotutoriales:
  - Funciones int() y float() en Python (Programa Resuelto)
  - Conversión de tipos en Python: int, float, str (Softwave Academy)
- Documentación: Funciones incorporadas y Tipos numéricos.

12.11 Comentarios

- Videotutoriales:
  - Programación en Python: comentarios (Programación ATS)
  - Docstrings: curso de introducción a Python (Santiago Hernández)
- Documentación: Comentarios en PEP 8 y PEP 257: convenciones de docstrings.
